

解析 s2 代码

会话 ID: 019f461f-28ef-7b93-96d2-f67e52cae570 开始时间: 2026-07-09 11:04:48 消息数: 55

用户 2026-07-09 11:04:48

详细解析s2下面的代码

助手 2026-07-09 11:04:55

我先把`s2`目录的结构和关键入口文件摸清楚，再按“整体流程 -> 各文件职责 -> 关键函数/类”的方式给你拆解。

助手 2026-07-09 11:05:11

目录里有两层：顶层是训练/调参/复现实验脚本和一批 YAML 配置，核心可复用代码在`s2/stage2/`包里。我先抓函数和类的索引，这样后面解析能对上具体文件与行号。

助手 2026-07-09 11:05:26

`README\_stage2.md`说明得比较明确：Stage2 是把 Stage1 导出的每条 flow embedding `z\_i` 冻结下来，再按时间/主机关系拼成 inter-flow context sequence，训练第二阶段分类器。接下来我读核心数据管线和模型代码。

助手 2026-07-09 11:05:40

数据部分的主线已经清楚了：`data\_io`负责把 Stage1 的 npz 和元数据拼齐并按时间排序；`context`一次扫描构造每条 flow 的上下文索引；`dataset`再把索引转成可 batch 的张量。现在看模型层和训练层。

助手 2026-07-09 11:05:53

模型这里不是单一 Transformer，而是保留了多种实验分支：无上下文 MLP、LSTM、普通 Transformer、残差上下文 Transformer，以及几个 target-query attention 变体。target-query 文件比较关键，因为 0707/0708 的配置看起来主要在跑这些新模型。

助手 2026-07-09 11:06:19

训练器承担了很多“实验工程”的细节：类别不平衡、采样器、AMP、阈值搜索、early stopping、保存预测与指标。我会把这部分作为代码执行流程的核心来讲。

助手 2026-07-09 11:06:33

我还注意到一个实现细节：`Stage2Trainer.fit()`标注返回`Dict`，但从代码看训练循环结束后没有显式`return`，主脚本也不依赖返回值，所以这是“注解不准”而不是当前流程的致命问题。下面我看入口脚本，确认参数怎么进入整个流程。

助手 2026-07-09 11:06:51

入口脚本 `run\_stage2.py`

基本就是把配置覆盖、数据准备、上下文构造、模型构建、训练、最终评估串起来。`run\_stage2\_tuned.py` 名字像调参后的入口，我再快速比较它和主入口的差异。

助手 2026-07-09 11:07:04

配置文件说明实验一直在围绕几个轴变化：上下文来源 (time/source/destination/endpoint/dual source-destination)、上下文隔离策略、模型类型、类别不平衡策略、阈值目标。最后我读调参和复现实验脚本，给你归类它们各自解决什么问题。

助手 2026-07-09 11:07:20

有个小发现：`tune\_stage2\_optuna.py` 里 `SharedStage2Dataset.\_\_getitem\_\_` 看起来缩进到了 `\_\_len\_\_` 里面，这会让这个 Dataset 在实际取样时没有可用的 `\_\_getitem\_\_`。如果你后面要跑 Optuna 脚本，这块值得优先修。

助手 2026-07-09 11:07:31

`verify\_reproducibility.py`

是另一类脚本：不是训练新模型，而是逐项检查数据加载、split、上下文、初始化、DataLoader、训练 determinism 等是否稳定。我只需要读它的入口和主要报告逻辑，不需要逐行展开 2000 行。

助手 2026-07-09 11:08:01

我再扫一下所有 YAML 的关键字段，方便把“这些配置分别在实验什么”讲清楚，不然只看文件名会有点迷雾。

助手 2026-07-09 11:08:57

下面是 `s2` 代码的整体解析。我没有修改文件，只做了阅读和梳理。

****整体定位****

`s2` 是 Stage2 inter-flow context 模型：它不再直接处理 packet，而是读取 Stage1 导出的每条 flow embedding `z\_i`，再根据时间、source、destination、endpoint 等关系为每条目标 flow 构造上下文序列，最后训练一个二阶段分类器。

主流程在 [run_stage2.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/run_stage2.py:470):

1. 读取 YAML 配置并应用命令行覆盖。
2. 从 Stage1 输出加载 `train/val/test` embeddings。
3. 按时间戳排序 flow，保证上下文是时间顺序。
4. 构造每条 flow 的 context indices。
5. 构造 `Stage2Dataset`。
6. 根据 `model.model\_type` 创建模型。
7. 用 `Stage2Trainer` 训练、保存 best checkpoint、最终评估并导出指标和预测。

****核心数据流****

配置由 [config.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2/config.py:8) 管理。`DEFAULT\_CFG` 分为四块：`data`、`context`、`model`、`training`。`load\_config()` 会用 YAML 覆盖默认配置，`deep\_update()` 是递归合并。

数据加载在

[data_io.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2/data_io.py:9)。它要求 Stage1 的`.npz`至少包含：

```
flow_id
label
z
```

同时需要`flow\_start\_timestamp\_us`、`source\_id`、`destination\_id`，这些可以在`.npz`里，也可以通过`stage1\_flow\_metadata.csv`合并。随后`prepare\_sorted\_stage2\_data()`会按`(timestamp, flow\_id)`稳定排序，并同步重排`z\_sorted`。

上下文构造在 [context.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2/context.py:12)。`ContextIndexBuilder`按时间从前往后扫描，不会在构造当前样本上下文前把当前 flow 放进历史，因此避免 future/self 泄漏。支持：

- `time\_only`：取全局最近历史 flow。
- `source\_host`：取相同 source 的历史 flow。
- `destination\_host`：取相同 destination 的历史 flow。
- `endpoint`：取 endpoint 相关历史 flow。
- `source\_destination`：分别构造 source 分支和 destination 分支，返回 dict。

`context\_policy` 控制上下文跨 split 的规则：

- `online`：所有更早 flow 都可作为上下文。
- `split\_isolated`：只使用同 split 的历史 flow。
- `train\_only\_for\_eval`：val/test 只看 train 历史。

****Dataset 与 batch****

[dataset.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2/dataset.py:10) 的`Stage2Dataset`接收排序后的`meta\_df`、`z\_sorted`、`context\_indices`，但每个 dataset 只暴露某个 split 的目标样本。

普通上下文返回：

```
context_z: [L, input_dim]
mask: [L]
relation_features: [L, 4]
label / flow_id / row_idx
```

dual source-destination 模式返回：

```
source_context_z / source_mask
destination_context_z / destination_mask
```

`stage2\_collate\_fn()`会左填充到固定长度`context.window\_size`：有效 token 放在右侧，所以“当前 flow”通常是最后一个 valid token。这一点后面模型里的`last\_valid\_token()`会依赖。

****模型层****

模型工厂在 [model.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2/model.py:796)，根据`model.model\_type`选择模型。

主要模型有：

- `no\_context\_mlp`：[Stage2NoContextMLP](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2/model.py:211)

只取当前 flow 的 Stage1 embedding 做分类，是无上下文 baseline。

- `lstm`: [Stage2LSTM](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2/model.py:265)
先把左填充序列压缩成右填充，再用 LSTM 编码上下文，支持 `last/mean/attention` pooling。

- `transformer`: [Stage2Transformer](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2/model.py:531)

普通 TransformerEncoder。可选正弦位置编码，`position\_mode` 支持 `age/order/absolute`。最后用 `last/mean/attention` pooling 分类。

- `residual\_transformer`: [Stage2ResidualContextTransformer](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2/model.py:630)

设计更保守：先用当前 flow 得到 `base\_logits`，再用历史上下文生成一个 gated `delta\_logits`，最终：

```
logits = base_logits + context_scale * gate * delta_logits
```

- target-query 系列在

[target_query.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2/target_query.py:29):

当前 flow 作为 query，历史 flow 作为 key/value。相比普通 Transformer，它更明确地建模 “当前 flow 应该关注哪些历史 flow”。

其中：

- `Stage2TargetQueryGatedAttention`：把上下文 summary 注入当前表示。

- `Stage2TargetQueryResidualAttention`：只让上下文修正 logits，更稳。

- `Stage2RelationAwareAttention`：额外用 `same\_source/same\_destination/same\_endpoint/age` 给 attention score 加 bias。

- `Stage2SourceDestinationAttention`：source history 和 destination history 两条分支分别 attention，再分别产生 gated residual logits。

****训练与评估****

训练器在 [trainer.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2/trainer.py:502)。

关键点：

- `make\_loss\_fn()` 支持 CE 和 focal loss，并可启用 class weight。

- `make\_train\_loader()` 支持 `WeightedRandomSampler`，可以用 `sampler\_pos\_fraction` 控制正样本采样比例。

- `PrecisionRegularizedLoss` 可额外惩罚负样本被预测成 class 1 的高概率，用来压 false positive。

- 训练使用 AdamW、warmup + cosine scheduler、AMP、gradient clipping。

- 每个 epoch 在 val 上评估，并可做 threshold search。

- best checkpoint 保存为 `stage2\_best\_model.pt`。

- 最终会输出：

- `stage2\_metrics.json`

- `stage2\_epoch\_metrics.csv`

- `stage2\_predictions\_train.csv`

- `stage2\_predictions\_val.csv`

- `stage2\_predictions\_test.csv`

- `stage2\_run\_summary.txt`

阈值搜索在

[trainer.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2/trainer.py:357)，可以优化 `f1\_label1`、`macro\_f1`、`precision\_at\_recall`、`recall\_at\_precision`、`precision\_lcb\_at\_recall` 等目标。

****调参与复现脚本****

[tune_transformer_hyperparams.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/tune_transfor

mer_hyperparams.py:275) 是网格/随机组合式调参脚本，会完整调用 `Stage2Trainer` 跑每组配置并汇总 CSV/JSON。

[tune_stage2_optuna.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/tune_stage2_optuna.py:779) 是 Optuna 版本，目标是 Colab/GPU 上更快搜索 Transformer 超参。它会只加载一次 Stage1 数据，并缓存不同 `window\_size` 的 context。

[verify_reproducibility.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/verify_reproducibility.py:1386) 是复现性检查套件，检查数据加载、context 构造、split、模型初始化、CPU/GPU 一致性、DataLoader 顺序、短训练 determinism，并生成报告。

****我看到的几个风险点****

1. [tune_stage2_optuna.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/tune_stage2_optuna.py:99) 里 `SharedStage2Dataset.\_\_getitem\_\_` 看起来缩进到了 `\_len\_\_` 内部，这会导致 Dataset 实际没有类级别的 `\_getitem\_\_`，Optuna 脚本运行时大概率会在 DataLoader 取样时报错。

2. [tune_transformer_hyperparams.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/tune_transformer_hyperparams.py:441) 的指标列表里 `"weighted\_recall"` 后面少了逗号，会和 `"f1\_label1"` 字符串拼接成一个错误 key，导致部分指标提取不到。

3. [trainer.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/stage2/trainer.py:548) 的 `fit()` 标注返回 `Dict[str, Any]`，但实际没有显式返回值。当前主流程不依赖这个返回值，所以不是阻塞问题，只是类型注解不准确。

总体来说，`s2` 已经是一个比较完整的 Stage2

实验框架：数据加载、上下文构造、模型变体、训练、阈值优化、调参和复现检查都有了。真正的核心路径是：

```
run_stage2.py
-> config.load_config
-> data_io.prepare_sorted_stage2_data
-> context.ContextIndexBuilder.build
-> dataset.Stage2Dataset + stage2_collate_fn
-> model.build_stage2_model
-> trainer.Stage2Trainer.fit
-> trainer.final_evaluate_and_save
```

用户 2026-07-09 11:09:33

stage1方案C的运行结果：{

```
"best_model_path": "/content/drive/MyDrive/s1/0704/0704C_ar002_et12_20260511_001/seqLen128heads
tage1_best_model.pt",
"best_epoch": 46,
"best_score": 0.9048145780941358,
"test_metrics": {
"accuracy": 0.9834202417634511,
"macro_precision": 0.8989505209974933,
"macro_recall": 0.8918983147078002,
"macro_f1": 0.8953896222507791,
"weighted_precision": 0.9832784957495156,
"weighted_recall": 0.9834202417634511,
"weighted_f1": 0.9833463190159758,
"precision_label1": 0.8069464544138929,
"recall_label1": 0.7920454545454545,
"f1_label1": 0.7994265232974911,
```

```

"auc": 0.9890371580790593,
"pr_auc": 0.8762558340285889,
"per_class_f1": {
  "0": 0.9913527212040671,
  "1": 0.7994265232974911
},
"confusion_matrix": [
  [
    80193,
    667
  ],
  [
    732,
    2788
  ]
],
"num_classes": 2,
"num_samples": 84380,
"loss": 0.06821018314045486,
"threshold": 0.6840000000000005
}
}

```

详细解析

助手 2026-07-09 11:10:21

这是一个很强的 Stage1 结果，尤其在类别极不平衡的 NIDS 场景下表现不错。核心结论先说：**方案 C 对正常流量识别非常稳，对恶意流量的 precision 已经较高，但 recall 仍是主要瓶颈，大约漏掉 20.8% 的恶意流。**

****混淆矩阵解析****

```

TN = 80193
FP = 667
FN = 732
TP = 2788

```

测试集共 `84380` 条 flow，其中：

```

: 80860      95.83%
: 3520       4.17%

```

这是明显类别不平衡数据，所以 `accuracy=0.9834` 虽然很高，但不能单独作为主要判断标准。更应该看 `precision\_label1 / recall\_label1 / f1\_label1 / PR-AUC`。

****恶意类检测能力****

恶意类，也就是 label 1：

```

precision_label1 = 0.8069
recall_label1    = 0.7920
f1_label1        = 0.7994

```

含义是：

- 模型预测为恶意的流量里，约 `80.7%` 真的恶意。
- 所有真实恶意流量里，模型抓到了约 `79.2%`。
- 还有 `732 / 3520 = 20.8%` 的恶意流被漏掉。

也就是说，方案 C

已经不是“只会识别正常类”的模型，它确实学到了恶意流特征；但如果用于入侵检测，**FN=732 这个漏报量仍然值得重点优化**。

****误报情况****

正常类误报为恶意：

```
FP = 667
FPR = 667 / 80860 = 0.825%
```

误报率不到 1%，这很好。换句话说，每 10000 条正常 flow 大约误报 82 条。对于 NIDS 来说，这个误报水平不算高，说明模型不是靠疯狂报警换 recall。

但代价是 recall 只有 `79.2%`。如果你的目标是“尽量不要漏攻击”，可以考虑降低 threshold；如果目标是“报警必须比较可信”，当前 threshold 还不错。

****阈值 threshold = 0.684****

这个阈值明显高于默认 `0.5`。说明训练/验证阶段自动阈值选择更偏向减少误报、提高 precision 或 F1。

当前 threshold 下：

```
: TP + FP = 2788 + 667 = 3455
: TP + FN = 2788 + 732 = 3520
```

预测恶意数量和真实恶意数量很接近，但并不是同一批样本完全对齐，所以有 667 个误报和 732 个漏报。

如果降低 threshold：

```
recall_label1
FN
precision_label1
FP
```

如果提高 threshold：

```
precision_label1
FP
recall_label1
FN
```

****AUC 与 PR-AUC****

```
auc = 0.9890
pr_auc = 0.8763
```

`AUC=0.989` 很高，说明模型整体排序能力很强：恶意样本通常能被赋予比正常样本更高的恶意概率。

`PR-AUC=0.876` 在恶意类只有 4.17% 的情况下也很不错。PR-AUC 比 ROC-AUC 更适合看不平衡数据，所以这个值更能说明方案 C 对少数类确实有效。

不过注意：高 AUC/PR-AUC 不等于某个固定 threshold 下完美。你现在的 `f1\_label1=0.7994` 说明模型概率排序好，但最终报警点仍然有优化空间。

****宏平均与加权平均****

```
macro_f1 = 0.8954
weighted_f1 = 0.9833
```

`weighted\_f1` 很高，主要因为正常类数量太多，正常类 F1 又高达：


```
class 0 f1 = 0.9914
class 1 f1 = 0.7994
```

所以更建议你重点报告：

```
macro_f1
f1_label1
precision_label1
recall_label1
pr_auc
confusion_matrix
```

这些比 accuracy 和 weighted_f1 更能体现攻击检测能力。

****训练状态****

```
best_epoch = 46
best_score = 0.9048
loss = 0.0682
```

`best\_epoch=46` 说明最优模型不是最后一轮，而是在第 46 epoch

验证指标达到最佳后保存的。`best\_score=0.9048` 应该是训练配置里用于选择 best model 的验证集指标，不一定等于这里的 test `macro\_f1` 或 `f1\_label1`。测试集上的恶意类 F1 是 `0.7994`，说明验证到测试存在一定下降，但整体仍然稳定。

****总体评价****

方案 C 的 Stage1 可以作为很强的 flow embedding 生成器：

```

:
-          FPR      0.825%
-    precision    80.7%
- AUC/PR-AUC
-    Stage2      embedding

:
-    recall    79.2%      20.8%
-    threshold
```

结合你前面的 `s2` 代码，这个 Stage1 结果非常适合继续进入 Stage2：Stage2 的作用就是利用 inter-flow context，让那些单条 flow embedding 不够明显的恶意样本，通过相邻时间、同 source、同 destination 或 endpoint 历史行为被补救回来。重点可以观察 Stage2 是否能把 `FN=732` 降下来，同时不要让 `FP=667` 大幅上升。

用户 2026-07-09 12:08:27

我stage2要跑出什么结果，才能证明stage2是有用的？

助手 2026-07-09 12:08:58

要证明 Stage2 有用，核心不是看 `accuracy`，而是要证明：

****Stage2 在使用 inter-flow context 后，比 Stage1 单条 flow 判断更好，尤其能减少漏报，同时误报不明显失控。****

你的 Stage1 方案 C 测试集基线是：

```
precision_label1 = 0.8069
recall_label1    = 0.7920
f1_label1        = 0.7994
macro_f1         = 0.8954
pr_auc           = 0.8763
FP = 667
```


FN = 732

所以 Stage2 最好至少做到下面几类结果之一。

****最有说服力的结果****

如果 Stage2 在同一个 test set 上达到类似：

```
f1_label1  > 0.82    0.83
macro_f1    > 0.90
pr_auc      > 0.89
recall_1    0.792
FP
```

那就能比较清楚地说明 Stage2 有价值。

更具体一点：如果 Stage2 把恶意类 recall 从 `0.792` 提到 `0.84`，意味着 FN 大约从 `732` 降到：

$3520 * (1 - 0.84) = 563$

也就是少漏报约 `169` 条恶意 flow。只要 FP 不从 `667` 暴涨到几千，这就是很有意义的改进。

****最低限度的有效证明****

至少要满足：

```
f1_label1    1.5    2
pr_auc
FN
FP
```

比如：

Stage1:
F1_1 = 0.7994, PR-AUC = 0.8763, FP=667, FN=732

Stage2:
F1_1 >= 0.815
PR-AUC >= 0.885
FN <= 650
FP 800~900

这种结果可以说 Stage2 有一定帮助，但说服力还不算特别强。

****强证明****

更理想的是：

Stage2:
precision_label1 >= 0.80
recall_label1 >= 0.84
f1_label1 >= 0.82~0.84
pr_auc >= 0.89~0.90
FN 732
FP 667

如果能做到 “recall 提高，precision 基本不降”，那就是非常漂亮的 Stage2 结果。

****必须做的对照实验****

为了证明是 “上下文” 有用，而不是训练技巧有用，需要比较：

1. Stage1
2. Stage2 no_context_mlp
3. Stage2 time_only
4. Stage2 source_host / destination_host / endpoint
5. dual_host_attention source_destination

最关键的是：

```
Stage2 context model > Stage2 no_context_mlp
```

因为 `no\_context\_mlp` 也使用 Stage1 embedding。如果 context 模型赢了 no-context baseline，才能说明 inter-flow context 真正在贡献信息。

****不要只看这些指标****

下面这些不能单独证明 Stage2 有用：

```
accuracy
weighted_f1
loss
```

因为你的数据恶意类只有约 `4.17%`，正常类太多。accuracy 和 weighted_f1 很容易被正常类支配。

你论文/实验里应该重点报告：

```
precision_label1
recall_label1
f1_label1
macro_f1
pr_auc
auc
confusion_matrix
threshold
```

****一句话判断标准****

如果 Stage2 能在严格无泄漏的 test set 上，比 Stage1 的 `f1\_label1=0.7994 / pr\_auc=0.8763 / FN=732` 明显更好，并且比 `no\_context\_mlp` 更好，就可以证明 Stage2 是有用的。

最有力的目标是：****Stage2 显著降低 FN，同时 FP 不明显爆炸。****

用户 2026-07-09 12:15:04

Files mentioned by the user:

```
## [INFO] Stage2 ----- seed: 130 [INFO] 随机种子已设置: 130 [INFO] Stage2 ----- d...:
C:\Users\XiaoyanXiong\codex/attachments/e144619a-cc04-4b54-b503-4944798eb095/pasted-text.txt
```

My request for Codex:

```
!python /content/drive/MyDrive/s2/run_stage2.py \
--stage1_dir /content/drive/MyDrive/s1/0704/0704C_ar002_et12_20260511_001 \
--config
/content/drive/MyDrive/s2/0706ar002_et12_20260511_001/timeOnly_mlp/stage2_config_used.yaml \
--out_dir /content/drive/MyDrive/s2/0706ar002_et12_20260511_001/timeOnly_mlp \
--context_method time_only \
--context_policy online \
--window_size 128 \
--seed 130 \
--epochs 500 \
--batch_size 128
```

运行的结果。详细解析

助手 2026-07-09 12:15:10

我先读取你贴出来的运行日志/结果文件，确认里面的最终 metrics、阈值和混淆矩阵，再结合你前面的 Stage1 方案 C 基线一起对比分析。

助手 2026-07-09 12:15:51

这次结果的关键结论是：

****这不是一个真正使用上下文的 Stage2，而是 `no\_context\_mlp` baseline。****

虽然命令里用了：

```
--context_method time_only
--window_size 128
```

但日志里模型是：

```
model_type = no_context_mlp
Stage2NoContextMLP
```

所以它构造了 time-only context，但模型实际只取“当前 flow 的 Stage1 embedding”做分类，****没有利用前 127 条历史 flow****。因此这次结果不能证明 inter-flow context 有用，只能证明：****在 Stage1 embedding 上再训练一个 MLP 分类头，会产生什么效果。****

****测试集结果****

Stage2 no-context MLP 的 test 结果是：

```
accuracy      = 0.9822
macro_f1      = 0.8929
precision_label11 = 0.7645
recall_label11  = 0.8281
f1_label11     = 0.7950
auc           = 0.9891
pr_auc        = 0.8755
threshold     = 0.6039
```

混淆矩阵：

```
TN = 79962
FP = 898
FN = 605
TP = 2915
```

也就是说：

- 真实恶意流量一共 `3520` 条。
- 模型抓到 `2915` 条。
- 漏掉 `605` 条。
- 正常流量误报 `898` 条。

****和 Stage1 方案 C 对比****

Stage1 方案 C：

```
precision_label11 = 0.8069
recall_label11    = 0.7920
f1_label11        = 0.7994
macro_f1          = 0.8954
pr_auc            = 0.8763
FP = 667
FN = 732
```

这次 Stage2 no-context MLP：

```
precision_label11 = 0.7645
recall_label11    = 0.8281
f1_label11        = 0.7950
macro_f1          = 0.8929
pr_auc            = 0.8755
FP = 898
```

FN = 605

变化是:

```
recall_label1: +0.0361
FN: 732 -> 605      127      flow

precision_label1: -0.0425
FP: 667 -> 898      231      flow

f1_label1: 0.7994 -> 0.7950
macro_f1: 0.8954 -> 0.8929
pr_auc: 0.8763 -> 0.8755
```

所以这次模型的行为很清楚: **它用更多误报换来了更高召回。 **

****怎么评价这次结果****

如果你的目标是“尽量少漏报攻击”，这次结果有价值，因为:

```
FN      127
recall   79.2%      82.8%
```

但如果你的目标是证明整体分类能力提升，这次结果不够强，因为:

```
f1_label1
macro_f1
precision
pr_auc
```

因此不能说这个 Stage2 明显优于 Stage1，只能说它更偏向 recall。

****阈值解释****

Stage1 的 threshold 是:

```
0.684
```

这次 Stage2 no-context MLP 的 threshold 是:

```
0.6039
```

阈值更低，所以模型更容易判为恶意。这也是为什么:

```
recall
FN
FP
precision
```

这不一定是模型表示能力变强，也可能主要是分类头和阈值选择更激进。

****验证集和测试集差异****

验证集表现很好:

```
val f1_label1 = 0.8219
val precision = 0.8144
val recall    = 0.8294
val pr_auc    = 0.9024
```

但测试集:

```
test f1_label1 = 0.7950
test precision = 0.7645
test recall    = 0.8281
test pr_auc    = 0.8755
```

测试集 recall 基本保持住了，但 precision 明显下降，说明 test 上多了一些 false positives。这个现象说明: **阈值从 val 迁移到 test 是稳定的，但 test 分布上正常样本更容易被误报。 **

`threshold transfer gap = 0.000474` 很小, 说明 val 选出的 threshold 和 test oracle threshold 几乎一样。这点是好的: 不是阈值选择问题, 主要是模型本身在 test 上 precision 不够。

****最终结论****

这次运行结果可以这样写:

	Stage2 no-context MLP		Stage1 embedding	
	recall	0.7920		0.8281
FN	732	605	127	
	precision	0.8069		0.7645
FP	667	898	f1_label1	0.7994 0.7950
			Stage1	

更重要的是:

```
model_type = no_context_mlp
            inter-flow context
```

下一步你应该跑真正使用上下文的模型, 比如:

```
model_type: transformer
context_method: time_only
```

或者更推荐:

```
model_type: target_query_gated
context_method: source_host / endpoint / source_destination
```

然后和这个 `no\_context\_mlp` 比。如果 context 模型能超过:

```
no_context_mlp test f1_label1 = 0.7950
no_context_mlp test pr_auc    = 0.8755
no_context_mlp FN              = 605
```

并且最好超过 Stage1 的:

```
Stage1 f1_label1 = 0.7994
Stage1 pr_auc    = 0.8763
```

那才是真正证明 Stage2 context 有用。

用户 2026-07-09 13:02:49

Files mentioned by the user:

```
## [INFO] Stage2 ----- seed: 42 [INFO] 随机种子已设置: 42 [INFO] Stage2 ----- dev...:
C:\Users\XiaoyanXiong\codex/attachments/c4bdc01e-ac53-41bd-8716-c8bae339c836/pasted-text.txt
```

My request for Codex:

```
!python /content/drive/MyDrive/s2/run_stage2.py \
--stage1_dir /content/drive/MyDrive/s1/0704/0704C_ar002_et12_20260511_001 \
--config /content/drive/MyDrive/s2/0709ar002_et12_20260511_001/LSTM/stage2_lstm_context.yaml \
--out_dir /content/drive/MyDrive/s2/0709ar002_et12_20260511_001/LSTM \
--context_method source_host \
--context_policy online \
--cls_head 3 \
--window_size 128 \
--epochs 500 \
--batch_size 128
```

运行结果。

详细解析结果

助手 2026-07-09 13:02:56

我先把这次 LSTM 运行日志读出来，重点抓最终 test metrics、best epoch、阈值和混淆矩阵，再和 Stage1 方案 C、上一轮 no_context MLP 做对比。

助手 2026-07-09 13:03:45

这次运行的是**真正用了上下文的 Stage2 模型**：

```
context_method = source_host
model_type     = lstm
window_size    = 128
context_policy = online
pooling        = last
cls_head       = 3
```

也就是说，每条目标 flow 会看同一个 `source\_id` 的历史 flow，最多前 127 条，再加当前 flow 本身，然后用 LSTM 编码这个 source-host 行为序列。

****最终 Test 结果****

```
accuracy      = 0.9822
macro_f1      = 0.8944
precision_label11 = 0.7583
recall_label11  = 0.8423
f1_label11     = 0.7981
auc           = 0.9898
pr_auc        = 0.8804
threshold     = 0.6287
```

混淆矩阵：

```
TN = 79915
FP = 945
FN = 555
TP = 2965
```

解释一下：

- 真实恶意 flow 一共 `3520` 条。
- LSTM 抓到了 `2965` 条。
- 漏掉 `555` 条。
- 正常 flow 误报了 `945` 条。

所以它的主要特点非常明确：****召回更高，漏报更少，但误报也更多。****

****和 Stage1 方案 C 对比****

Stage1 方案 C：

```
precision_label11 = 0.8069
recall_label11    = 0.7920
f1_label11        = 0.7994
macro_f1          = 0.8954
pr_auc            = 0.8763
FP = 667
FN = 732
```

Stage2 LSTM source_host：

```
precision_label11 = 0.7583
recall_label11    = 0.8423
f1_label11        = 0.7981
macro_f1          = 0.8944
```

```
pr_auc          = 0.8804
FP = 945
FN = 555
```

变化是:

```
recall_label1: 0.7920 -> 0.8423      +5.03
FN:           732 -> 555              177      flow

precision:     0.8069 -> 0.7583      -4.86
FP:           667 -> 945              278      flow

f1_label1:     0.7994 -> 0.7981
macro_f1:      0.8954 -> 0.8944
pr_auc:        0.8763 -> 0.8804
auc:           0.9890 -> 0.9898
```

所以结论不是“全面超过 Stage1”，而是:

****Stage2 LSTM 确实提升了恶意流量的排序能力和召回能力，但它用更多误报换来了更少漏报，最终 F1 基本没有超过 Stage1。****

****和 no-context MLP 对比****

上一次 no-context MLP:

```
precision_label1 = 0.7645
recall_label1    = 0.8281
f1_label1        = 0.7950
pr_auc           = 0.8755
FP = 898
FN = 605
```

这次 LSTM source_host:

```
precision_label1 = 0.7583
recall_label1    = 0.8423
f1_label1        = 0.7981
pr_auc           = 0.8804
FP = 945
FN = 555
```

相比 no-context:

```
recall    +1.42
FN        50
f1_label1 +0.31
pr_auc    +0.49
FP        47
precision
```

这个对比很重要。它说明：****source_host LSTM 的上下文是有一点帮助的****，因为它比不用上下文的 MLP 多抓到了一些恶意 flow，并且 PR-AUC 更高。但提升幅度还不够大，不算强证明。

****上下文本身是否有信息****

从 context diagnostics 看，source_host 上下文是有结构差异的:

```
test label 0 history_len mean = 98.01
test label 1 history_len mean = 117.88
```

恶意类平均有更长的 source history。这说明恶意 flow 往往来自更活跃或重复出现的 source，这正是 Stage2 source_host context 可以利用的信号。

no-history rate 也很低:

```
test label 0 no-history = 2.23%
test label 1 no-history = 1.59%
```


所以绝大多数样本都有可用历史。Stage2 不是因为缺少上下文而失败，而是当前 LSTM 对这些上下文的利用还比较有限。

****训练过程解析****

最佳模型出现在：

```
best_epoch = 14
early stopping at epoch = 29
metric_for_best = pr_auc
```

训练前几轮提升很快：

```
epoch 1: val_pr_auc = 0.6124
epoch 2: val_pr_auc = 0.8939
epoch 7: val_pr_auc = 0.9070
epoch 14: val_pr_auc = 0.9078
```

之后训练 loss 继续下降，但 val 指标没有稳定提升，说明后面开始有轻微过拟合。`best\_epoch=14` 是合理的。

验证集表现：

```
val f1_label1 = 0.8243
val pr_auc     = 0.9079
```

测试集表现：

```
test f1_label1 = 0.7981
test pr_auc     = 0.8804
```

验证到测试有明显下降，尤其 precision 下降。这说明模型在测试集上产生了更多 false positives。

****阈值问题****

验证集自动选出的阈值是：

```
0.6287
```

在 test 上使用该阈值：

```
precision = 0.7583
recall    = 0.8423
f1         = 0.7981
```

测试集 oracle threshold 是：

```
0.7128
```

如果用 test oracle，结果会变成：

```
precision = 0.7984
recall    = 0.8020
f1         = 0.8002
FP = 713
FN = 697
```

这说明模型其实可以调到更接近 Stage1 的 precision/F1，但那会牺牲 recall。正式论文里不能用 test oracle，只能作为诊断：****当前 val 选择的阈值偏向高召回，导致 test FP 较多。****

****最终判断****

这次 Stage2 LSTM source_host 有价值，但证据强度是“中等偏弱”：

```
Stage2      :
- recall_label1 Stage1 0.7920 0.8423
- FN   732   555   177
- PR-AUC 0.8763 0.8804
- no-context MLP FN   50  PR-AUC
```

```

:
- precision    0.8069    0.7583
- FP      667    945
- f1_label1    Stage1 0.7994 -> 0.7981
- macro_f1    Stage1

```

可以这样总结:

Stage2 LSTM with source-host context improves malicious-flow recall and reduces false negatives, showing that inter-flow source context contains useful information. However, the improvement is mainly recall-oriented and comes with increased false positives; overall F1 remains comparable to, but not clearly better than, the Stage1 baseline.

如果想证明 Stage2 更强, 下一步目标应该是: 保持这次的高 recall, 同时把 FP 压下来。理想结果是:

```

recall_label1 >= 0.83
precision_label1 >= 0.79    0.80
f1_label1 > 0.81
pr_auc > 0.885
FP      800
FN      732

```

用户 2026-07-09 13:52:31

Files mentioned by the user:

```

## [INFO] Stage2 ----- seed: 130 [INFO] 随机种子已设置: 130 [INFO] Stage2 ----- d...:
C:\Users\XiaoyanXiong\codex/attachments/360804da-79eb-4f90-aac3-d4638dd7e9ad/pasted-text.txt

```

My request for Codex:

```

!python /content/drive/MyDrive/s2/run_stage2.py \
--stage1_dir /content/drive/MyDrive/s1/0704/0704C_ar002_et12_20260511_001 \
--config /content/drive/MyDrive/s2/0709ar002_et12_20260511_001/target_query_gated_v1/stage2_config_used.yaml \
--out_dir /content/drive/MyDrive/s2/0709ar002_et12_20260511_001/target_query_gated_v1/cls2 \
--context_method source_host \
--context_policy online \
--window_size 128 \
--seed 130 \
--epochs 100 \
--batch_size 128

```

运行结果。

详细解析

助手 2026-07-09 13:52:38

我来读取这次 `target\_query\_gated\_v1` 的日志, 重点看它是不是用了真正的 source-host context、最终 test 指标、阈值迁移, 以及和 Stage1 / no-context / LSTM 的差异。

助手 2026-07-09 13:53:31

这次结果是目前最有说服力的一次 Stage2: 它确实用了上下文, 而且 test 上 `f1\_label1`、`macro\_f1`、`pr\_auc` 都略高于 Stage1。提升幅度不大, 但方向是对的。

****这次模型配置****

```

context_method = source_host
context_policy = online

```

```
window_size      = 128
model_type       = target_query_gated
cls_head         = 2
seed             = 130
```

它不是普通 Transformer，也不是 LSTM，而是：

```
Stage2TargetQueryGatedAttention
```

含义是：当前 flow 作为 query，同 source 的历史 flow 作为 key/value，模型学习 “当前 flow 应该关注哪些历史 source 行为”，再通过 gate 决定上下文信息注入多少。

****最终 Test 结果****

```
accuracy         = 0.9829
macro_f1         = 0.8957
precision_label1 = 0.7783
recall_label1    = 0.8236
f1_label1        = 0.8003
auc              = 0.9896
pr_auc           = 0.8796
threshold        = 0.6336
```

混淆矩阵：

```
TN = 80034
FP = 826
FN = 621
TP = 2899
```

也就是说，3520 条恶意 flow 里抓到了 2899 条，漏掉 621 条。

****和 Stage1 方案 C 对比****

Stage1：

```
precision_label1 = 0.8069
recall_label1    = 0.7920
f1_label1        = 0.7994
macro_f1         = 0.8954
pr_auc           = 0.8763
FP = 667
FN = 732
```

这次 target-query Stage2：

```
precision_label1 = 0.7783
recall_label1    = 0.8236
f1_label1        = 0.8003
macro_f1         = 0.8957
pr_auc           = 0.8796
FP = 826
FN = 621
```

变化：

```
recall_label1: +3.15
FN: 732 -> 621      111      flow

precision_label1: -2.87
FP: 667 -> 826      159      flow

f1_label1: 0.7994 -> 0.8003
macro_f1: 0.8954 -> 0.8957
pr_auc: 0.8763 -> 0.8796
```

所以它比 Stage1 的优势是：****少漏报，排序能力略好，恶意类 F1 略高。****

代价是：****误报增加，precision 下降。****

****和 no-context MLP 对比****

no-context MLP:

```
precision_label1 = 0.7645
recall_label1    = 0.8281
f1_label1        = 0.7950
pr_auc           = 0.8755
FP = 898
FN = 605
```

target-query:

```
precision_label1 = 0.7783
recall_label1    = 0.8236
f1_label1        = 0.8003
pr_auc           = 0.8796
FP = 826
FN = 621
```

这里很关键: target-query 相比不用上下文的 MLP, **precision、F1、PR-AUC 都更好, 并且 FP 少了 72 条**。虽然 FN 多了 16 条, 但整体更平衡。

这能支持一个结论:

```
source-host context    Stage2
```

****和 LSTM source_host 对比****

LSTM:

```
precision_label1 = 0.7583
recall_label1    = 0.8423
f1_label1        = 0.7981
pr_auc           = 0.8804
FP = 945
FN = 555
```

target-query:

```
precision_label1 = 0.7783
recall_label1    = 0.8236
f1_label1        = 0.8003
pr_auc           = 0.8796
FP = 826
FN = 621
```

LSTM 更偏召回, 少漏报; target-query 更平衡, 误报少很多, F1 更高一点。可以说:

```
LSTM      target-query
```

****训练过程****

最佳 epoch 是:

```
best_epoch = 5
early stopping at epoch = 20
metric_for_best = pr_auc
```

验证集最优结果:

```
val f1_label1 = 0.8282
val pr_auc    = 0.9118
val threshold = 0.6336
```

测试集:

```
test f1_label1 = 0.8003
test pr_auc    = 0.8796
```

验证到测试下降比较明显，说明 test 分布更难。但测试结果仍略高于 Stage1。

****阈值诊断很重要****

正式使用验证集阈值 `0.6336` 时：

```
precision = 0.7783
recall    = 0.8236
f1        = 0.8003
FP = 826
FN = 621
```

测试集 oracle threshold 是 `0.6881`，仅作诊断，不能作为正式结果：

```
precision = 0.8123
recall    = 0.7940
f1        = 0.8030
FP = 646
FN = 725
```

这个很有意思：如果阈值稍微提高，模型可以做到比 Stage1 更高 precision、更高 F1、更低 FP，但 recall 会回到接近 Stage1。说明模型本身排序能力有提升，当前正式阈值只是偏向召回。

****最终判断****

这次结果可以作为 Stage2 有用的证据，但属于“小幅有效”，不是大幅碾压。

可以这样写：

```
Target-query gated Stage2 with source-host context slightly improves over the Stage1 baseline.
It increases malicious recall from 0.7920 to 0.8236, reduces false negatives from 732 to 621,
and improves class-1 F1 from 0.7994 to 0.8003 and PR-AUC from 0.8763 to 0.8796.
Compared with the no-context Stage2 MLP, it also improves F1 and PR-AUC while reducing false positives,
indicating that source-host inter-flow context contributes useful information.
```

最简洁结论：

target_query_gated	“Stage2”		
Stage1	no-context MLP	LSTM	
	threshold	precision regularization	FP

用户 2026-07-09 13:59:23

我知道我现在的代码没有出现data leakage,我想知道出现data leakage会是什么样的结果。请给我提共一个train,val,test出现泄露的stage2的代码。

助手 2026-07-09 13:59:38

我会给你做一个****独立的 leaky 实验脚本****，不碰你当前干净的 `run\_stage2.py` 主流程。这个脚本会明确标注“只用于演示 data leakage”，让 train/val/test 的上下文互相可见，甚至可以包含 future flows，这样你能看到泄露后指标通常会怎么虚高。

助手 2026-07-09 14:00:27

我会把泄露做成两个层级：默认是“上下文泄露”（每个 split 的样本都能看见 train/val/test 的过去和未来 flow）；可选再加 `--leak\_train\_on\_eval`，让训练集目标本身也包含 val/test，这就是非常强的评估污染。这样你能分别观察“轻度/重度 leakage”会怎样把指标抬高。

助手 2026-07-09 14:01:23

已经加了一个独立脚本 `s2/run\_stage2\_leaky.py`。我现在做一个语法检查，确保它至少能被 Python 正常解析，不会因为缩进/导入写错直接炸掉。

助手 2026-07-09 14:01:32

本机 PowerShell 里 `python` 不在 PATH，我换 Windows 的 `py` launcher 做同样的语法检查。

助手 2026-07-09 14:01:43

这里没有 `python/py` 命令，但之前环境里有用户本地 Python 路径。我用那个绝对路径检查一次。

助手 2026-07-09 14:01:49

绝对路径的 Python 被沙箱拦住了；我改用 Codex 自带的工作区运行时来做只读语法检查。

助手 2026-07-09 14:01:51

拿到了 Codex bundled Python 路径，继续检查脚本语法。

助手 2026-07-09 14:02:23

我已经新增了一个独立脚本：

[run_stage2_leaky.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/run_stage2_leaky.py)

它不会改你现在干净的 `run\_stage2.py`，专门用于“负面对照实验”：故意制造 train/val/test 泄露，看指标会怎么虚高。

****它制造两种泄露****

默认开启的是上下文泄露：

```
train/val/test    context
1. train/val/test split flow
2. flow          flow  future context
```

这会破坏真实评估，因为 test 样本在构造 context 时可以看到未来样本、val 样本、train 样本。

可选更强泄露：

```
--leak_train_on_eval
```

这个会让 Stage2 的训练目标样本直接包含原始 `train + val + test`，也就是模型训练时已经见过 test target。这种结果通常会非常虚高，只能用来演示“严重 leakage 长什么样”。

****Colab 运行示例：上下文泄露版****

```
!python /content/drive/MyDrive/s2/run_stage2_leaky.py \
  --stage1_dir /content/drive/MyDrive/s1/0704/0704C_ar002_et12_20260511_001 \
  --config /content/drive/MyDrive/s2/0709ar002_et12_20260511_001/target_query_gated_v1/stage2_config_used.
yaml \
  --out_dir /content/drive/MyDrive/s2/LEAKY/source_host_future_context \
  --context_method source_host \
  --context_policy online \
  --window_size 128 \
  --seed 130 \
  --epochs 100 \
  --batch_size 128
```

****Colab 运行示例：严重 train/val/test target 泄露版****

```
!python /content/drive/MyDrive/s2/run_stage2_leaky.py \
  --stage1_dir /content/drive/MyDrive/s1/0704/0704C_ar002_et12_20260511_001 \
  --config /content/drive/MyDrive/s2/0709ar002_et12_20260511_001/target_query_gated_v1/stage2_config_used.
yaml \
  --out_dir /content/drive/MyDrive/s2/LEAKY/source_host_train_on_eval \
  --context_method source_host \
  --context_policy online \
  --window_size 128 \
  --seed 130 \
  --epochs 100 \
  --batch_size 128 \
  --leak_train_on_eval
```

****你应该观察什么现象****

如果只是上下文泄露，通常会看到：

```
val/test pr_auc
val/test f1_label1
threshold transfer gap
test      val
```

如果用了 `--leak\_train\_on\_eval`，可能会看到更夸张的结果：

```
test f1_label1
test loss
test precision/recall
val/test  train
```

这种结果越漂亮，越说明它不可信。

****判断 leakage 的典型信号****

正常实验里，你现在 Stage2 大概在：

```
f1_label1: 0.795 ~ 0.800
pr_auc:    0.875 ~ 0.880
```

如果 leaky 版本突然变成类似：

```
f1_label1 > 0.85
pr_auc    > 0.92
FP/FN
val/test
```

那就是典型 data leakage 味道。漂亮得太顺滑，反而危险。

我也做了语法检查：`run\_stage2\_leaky.py` 可以正常通过 `py\_compile`。

用户 2026-07-09 14:25:43

详细解析代码，如何做到严重 train/val/test target 泄露版

助手 2026-07-09 14:26:18

我先把刚刚新增的 leaky 脚本按行号看一遍，然后专门拆解 `--leak\_train\_on\_eval` 是怎么让 train/val/test target 泄露进训练集的。

助手 2026-07-09 14:27:10

严重 `train/val/test target 泄露版` 的核心在这里：

[run_stage2_leaky.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/run_stage2_leaky.py:170)

它通过 `--leak\_train\_on\_eval` 参数触发。

****关键代码 1：命令行开关****

```
parser.add_argument(
    "--leak_train_on_eval",
    action="store_true",
    help="Also train the Stage2 classifier on train+val+test target rows.",
)
```

位置: [run_stage2_leaky.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/run_stage2_leaky.py:170)

意思是：只要运行时加上：

```
--leak_train_on_eval
```

脚本就会把原本的 `train/val/test` 目标样本全部放进训练集。

****关键代码 2：把所有 split 改成 train****

最严重泄露发生在这里：

```
train_meta = meta_df
if leak_train_on_eval:
    train_meta = meta_df.copy()
    train_meta["split"] = "train"
    print("[LEAKY] Training target rows include original train+val+test rows.")
```

位置: [run_stage2_leaky.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/run_stage2_leaky.py:218)

正常情况下，`meta\_df["split"]` 是：

```
train / val / test
```

而这段代码会复制一份 `meta\_df`，然后把所有行的 split 强行改成：

```
train
```

于是原来的 val 样本、test 样本，在训练数据里也变成了 train 样本。

****为什么这会造成 target 泄露****

你的 `Stage2Dataset` 是这样选目标样本的：

```
self.target_rows = self.meta_df.index[
    self.meta_df["split"] == target_split
].to_numpy(dtype=np.int64)
```

也就是说，传入：

```
target_split="train"
```

它就会选出所有 `split == "train"` 的行作为训练目标。

在 leaky 脚本里，执行了：

```
train_meta["split"] = "train"
```

所以：

```
train    -> train
val      -> train
test     -> train
```


最终训练集包含：

```
train + val + test
```

这就是最严重的 target leakage: **模型训练时已经见过 test 样本的 embedding 和 label.**

关键代码 3: 训练集用 leaky meta, 验证/测试仍用原 meta

```
return {
    "train": Stage2Dataset(
        meta_df_sorted=train_meta,
        ...
        target_split="train",
    ),
    "val": Stage2Dataset(
        meta_df_sorted=meta_df,
        ...
        target_split="val",
    ),
    "test": Stage2Dataset(
        meta_df_sorted=meta_df,
        ...
        target_split="test",
    ),
}
```

位置: [run_stage2_leaky.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/run_stage2_leaky.py:224)

这里非常关键:

- `train` dataset 用的是 `train\_meta`, 其中所有样本都被改成 train。
- `val/test` dataset 仍然用原始 `meta\_df`, 所以评估时看起来还是在 val/test 上评估。
- 但实际上, val/test 的目标样本已经在训练阶段出现过。

所以评估流程表面上像:

```
train on train
evaluate on val/test
```

但真实情况是:

```
train on train+val+test
evaluate on already-seen val/test
```

这就是严重泄露版。

再叠加上下文泄露

这个脚本还故意让 context 泄露。位置在:

[run_stage2_leaky.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/run_stage2_leaky.py:22)

正常 `ContextIndexBuilder` 是按时间向前扫描, 先构造当前样本 context, 再把当前样本加入 history, 所以不会看到未来。

但 leaky 版先把所有 row 全部放进 map:

```
for idx, (src, dst) in enumerate(zip(sources, destinations)):
    source_map[src].append(idx)
    dest_map[dst].append(idx)
    endpoint_map[src].append(idx)
```

位置: [run_stage2_leaky.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/run_stage2_leaky.py:109)

然后对每个目标样本，从完整的 source/destination/endpoint 列表里取邻近样本：

```
ctx = self._nearest_from_sorted(source_map[src], idx)
```

位置：[run_stage2_leaky.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/run_stage2_leaky.py:136)

这里没有过滤：

```
val
test
flow
```

所以每个样本的 context 可以包含：

```
train flow
val flow
test flow
future flow
```

这属于 context leakage。

****最严重泄露的完整运行命令****

```
!python /content/drive/MyDrive/s2/run_stage2_leaky.py \
--stage1_dir /content/drive/MyDrive/s1/0704/0704C_ar002_et12_20260511_001 \
--config /content/drive/MyDrive/s2/0709ar002_et12_20260511_001/target_query_gated_v1/stage2_config_used.
yaml \
--out_dir /content/drive/MyDrive/s2/LEAKY/source_host_train_on_eval \
--context_method source_host \
--context_policy online \
--window_size 128 \
--seed 130 \
--epochs 100 \
--batch_size 128 \
--leak_train_on_eval
```

****它会泄露什么****

这个命令同时造成两层泄露：

1. context leakage:
train/val/test + future flow
2. target leakage:
val/test label

第二种最严重，因为模型已经在训练阶段见过测试目标的：

```
z embedding
label
context
```

所以 test 指标会变得不可信。

****泄露后结果通常是什么样****

你可能会看到：

```
val/test f1_label1
val/test pr_auc
test loss
train/val/test
threshold transfer gap
FP FN
```

但这些都不能作为论文结果，只能作为 negative control，用来展示：

```
data leakage Stage2
```

一句话总结：

这个严重泄露版的关键就是 [第 220-221

行](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/run_stage2_leaky.py:220)：把 `meta\_df` 全部复制成 `train\_meta`，再把所有 `split` 改成 `train`，导致模型训练时直接看到了 val/test 的 target 样本和标签。

用户 2026-07-09 15:27:06

如何做到只泄露val和test的一半到train里面？我想做测试

助手 2026-07-09 15:27:23

可以，我会把 leaky 脚本扩展成“部分泄露”版本：`--leak\_train\_on\_eval --leak\_eval\_fraction 0.5` 时，只把原始 val 的 50% 和 test 的 50% 标成 train；剩下的一半仍然保持 val/test。这样训练集会混入一半验证/测试目标，评估集仍然是完整 val/test，用来观察“半泄露”会把指标抬高多少。

助手 2026-07-09 15:27:25

我会保持原来的全泄露行为不变：不传 `--leak\_eval\_fraction` 时默认还是 1.0；你传 0.5 就是半泄露。

助手 2026-07-09 15:27:45

代码已改好。我再跑一次语法检查，确认新增参数和函数签名都对上了。

助手 2026-07-09 15:28:06

为了避免“半 target 泄露”和“future/context 泄露”混在一起不好判断，我再加一个可选开关 `--clean\_context`：打开后，上下文构造使用你原本干净的 `ContextIndexBuilder`，只保留“val/test 一半进入 train”的 target 泄露。

助手 2026-07-09 15:28:18

现在脚本可以分别测两种情况：纯 target 半泄露，或者 target 半泄露叠加上文泄露。我再做一次语法检查。

助手 2026-07-09 15:28:49

已支持。现在

[run_stage2_leaky.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/run_stage2_leaky.py:170) 可以做到：

```
val    50% + test    50%    train
```

****推荐命令：只做 target 半泄露，context 保持正常****

```
!python /content/drive/MyDrive/s2/run_stage2_leaky.py \
--stage1_dir /content/drive/MyDrive/s1/0704/0704C_ar002_et12_20260511_001 \
--config /content/drive/MyDrive/s2/0709ar002_et12_20260511_001/target_query_gated_v1/stage2_config_used.
yaml \
--out_dir /content/drive/MyDrive/s2/LEAKY/half_val_test_to_train_clean_context \
--context_method source_host \
--context_policy online \
--window_size 128 \
```

```
--seed 130 \
--epochs 100 \
--batch_size 128 \
--leak_train_on_eval \
--leak_eval_fraction 0.5 \
--leak_eval_seed 130 \
--clean_context
```

这里最关键的是：

```
--leak_train_on_eval
--leak_eval_fraction 0.5
--clean_context
```

`--clean\_context` 表示：上下文仍然用原来的 `ContextIndexBuilder`，不额外制造 future-context leakage。这样你测到的主要就是 “val/test target 一半进入 train” 的影响。

****代码如何做到半泄露****

核心在

[run_stage2_leaky.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/run_stage2_leaky.py:218):

```
train_meta = meta_df.copy()
```

先复制完整 `meta\_df`，保持行号和 `z\_sorted/context\_indices` 对齐。不能直接过滤一半 rows，否则 `Stage2Dataset` 的 row index 会错位。

然后对 val/test 分别抽样：

```
rows = train_meta.index[train_meta["split"] == split].to_numpy(dtype=np.int64)
n_leak = int(round(len(rows) * leak_eval_fraction))
leaked_rows = rng.choice(rows, size=n_leak, replace=False)
train_meta.loc[leaked_rows, "split"] = "train"
```

位置：[run_stage2_leaky.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/run_stage2_leaky.py:225)

含义是：

```
train:      train
val:        50%   train
test:       50%   train
```

剩下没抽中的 val/test 仍然保持原 split。

最后构造 dataset 时：

```
"train": Stage2Dataset(meta_df_sorted=train_meta, target_split="train")
"val":   Stage2Dataset(meta_df_sorted=meta_df, target_split="val")
"test":  Stage2Dataset(meta_df_sorted=meta_df, target_split="test")
```

位置：[run_stage2_leaky.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/run_stage2_leaky.py:238)

所以训练时：

```
train dataset = train + 50% val + 50% test
```

评估时：

```
val dataset = val
test dataset = test
```

这意味着最终 val/test

指标是 “混合评估”：里面一半样本模型训练时见过，另一半没见过。因此指标会被部分抬高，但不会像 100% 泄露那么夸张。

****如果你想叠加 context 泄露****

去掉 `--clean\_context` 即可：

```
--leak_train_on_eval \  
--leak_eval_fraction 0.5 \  
--leak_eval_seed 130
```

这样会同时发生：

1. 50% val/test target train
2. context train/val/test + future flows

这个更脏，更适合看“泄露后指标会多离谱”。

****运行时看日志****

脚本会打印类似：

```
[LEAKY] Training target rows include original train plus sampled eval rows:  
fraction=0.500, leaked_counts={'val': ..., 'test': ...}
```

你的数据大概会泄露：

```
val:   42189      21094  
test: 84380      42190
```

这个版本我已经做过语法检查，可以正常解析。

用户 2026-07-09 16:48:18

上面的label是否也一起合并进去了？不会出现其他问题吧，都是一一对应的吗？

助手 2026-07-09 16:48:46

是的，**label 也一起进入 train 了**，而且是一一对应的。

原因是半泄露代码不是单独拼接 `z` 或单独拼接 label，而是直接复制完整的 `meta\_df`：

```
train_meta = meta_df.copy()
```

`meta\_df` 里面本来就包含：

```
flow_id  
label  
split  
source_id  
destination_id  
timestamp
```

然后代码只改 sampled val/test 行的 `split`：

```
train_meta.loc[leaked_rows, "split"] = "train"
```

它没有改 `label`，也没有改行顺序。所以这些被泄露进去的 val/test 样本会保留原始 label。

之后 `Stage2Dataset` 会这样取训练标签：

```
self.target_rows = self.meta_df.index[self.meta_df["split"] == target_split]  
self.labels = self.meta_df.loc[self.target_rows, "label"].to_numpy(...)
```

所以当 `target\_split="train"` 时，训练集拿到的是：

```
train  label  
+      50% val  label  
+      50% test  label
```

****——对应关系也不会乱****

不会乱的关键原因是：

```
train_meta    meta_df
train_meta    meta_df
train_meta    index
z_sorted
context_indices
```

所以每一行仍然满足：

```
row_idx -> meta_df/ train_meta    label
row_idx -> z_sorted[row_idx]
row_idx -> context_indices[row_idx]
```

也就是说，被泄露的 test 第 i 行进入 train 后，它训练时用的仍然是：

```
z_sorted[i]
label[i]
context_indices[i]
flow_id[i]
```

不会出现 label 和 embedding 错位。

****会不会有重复样本？****

在 train dataset 内部不会重复。因为代码是把一部分 val/test 的 `split` 改成 `train`，不是 append 一份副本。

但从实验意义上看，会出现：

```
val/test          val/test
```

这正是我们想制造的 target leakage。

****注意一个小点****

如果你想只测试 “target 一半泄露”，建议用：

```
--clean_context
```

并且更严格的话可以加：

```
--context_policy split_isolated
```

这样就尽量避免 context 也额外泄露。否则如果用 `context\_policy online`，上下文可能仍然跨 split 使用历史 flow，这会混入另一种 context leakage，不利于单独观察 target leakage 的影响。

简短结论：****label 会一起进 train，embedding/label/context 都是一一对应的；唯一 “不正常” 的地方正是 val/test 的一半目标样本和标签被训练集看到了。****

用户 2026-07-10 08:39:52

```
--leak_train_on_eval \
--leak_eval_fraction 0.5 \
--leak_eval_seed 130 \
--clean_context
```

详细解析这几个

助手 2026-07-10 08:40:15

这 4 个参数合起来的意思是：

```
“target” context
```

也就是：**把 val/test 各随机抽 50% 放进 train 训练，但上下文构造仍然用正常逻辑。**

```
**`--leak_train_on_eval`**
```

这是总开关。

不加它时，脚本不会把 val/test 样本放进训练集。加了它以后，才会执行这段逻辑：

```
train_meta = meta_df.copy()
...
train_meta.loc[leaked_rows, "split"] = "train"
```

也就是说，它允许脚本把一部分原本属于 `val` 和 `test` 的目标样本改成 `train`，让模型训练时看到它们的：

```
embedding
label
flow_id
context
```

这是 target leakage 的核心。

```
**`--leak_eval_fraction 0.5`**
```

这个控制泄露比例。

```
0.5 = val      test
1.0 = val/test  train
0.25 = val/test
```

你的数据里：

```
val  = 42189
test = 84380
```

所以 `0.5` 大约会泄露：

```
val  21094  train
test 42190  train
```

训练集会从原来的：

```
train = 295329
```

变成大约：

```
train = 295329 + 21094 + 42190 = 358613
```

注意：评估时 `val/test` 仍然按完整原始 val/test 评估，所以里面有一半样本是模型训练时见过的。

```
**`--leak_eval_seed 130`**
```

这个控制 “随机抽哪一半” 。

因为 `0.5` 不是取前一半，而是随机抽样：

```
rng = np.random.default_rng(130)
leaked_rows = rng.choice(rows, size=n_leak, replace=False)
```

所以 seed 固定为 `130` 后，每次运行都会抽到同一批 val/test 样本，结果更可复现。

如果你换成：

```
--leak_eval_seed 42
```

泄露数量还是 50%，但具体是哪一批 val/test 样本会变，结果可能略有不同。

```
**`--clean_context`**
```

这个非常重要。它表示：****不要使用 leaky context builder，改用原来的正常 ContextIndexBuilder。****

如果不加 `--clean_context`，脚本默认会让 context 也泄露：

```
context    train/val/test
context    future flow
```

这会把 target leakage 和 context leakage 混在一起，结果不好解释。

加了 `--clean_context` 后：

```
val/test target    train
```

也就是说，这个实验更干净：你观察到的指标虚高主要来自：

```
val/test    label
```

而不是来自 future context。

****合起来的实际效果****

这四个参数一起：

```
--leak_train_on_eval \
--leak_eval_fraction 0.5 \
--leak_eval_seed 130 \
--clean_context
```

等价于：

```
1.    val    50%    train
2.    test   50%    train
3.    train  train
4.    =    train + 50% val + 50% test
5.    =    val /    test
6.
```

所以最终 test 指标会有“半真半假”的味道：

```
test
test
```

如果这个实验结果明显高于正常 Stage2，就可以说明：****哪怕只有一半 test target 泄露进 train，也会把最终 test 指标抬高。****

用户 2026-07-10 09:05:04

现在这个 leak_eval_fraction 使随机的，我想知道按顺序取该如何修正代码？详细解析

助手 2026-07-10 09:05:17

我会把“随机抽 50%”改成可选策略：默认仍然随机；如果你传 `--leak_eval_selection first`，就按当前 `meta_df` 的顺序取每个 split 的前 50%。因为 `meta_df` 已经在 `prepare_sorted_stage2_data()` 里按时间排序，所以这里的“first”实际就是按时间顺序取该 split 最早的一半。

助手 2026-07-10 09:05:26

我看到了当前实现确实固定使用 `rng.choice(...)`。接下来我加一个 `--leak_eval_selection {random,first,last}` 参数，并把选择逻辑独立出来，避免以后再扩展时把这段越写越绕。

助手 2026-07-10 09:05:45

修改完成了。`first` 会按排序后的行号取前半, `last` 取后半; `random` 仍保留旧行为。我现在做语法检查。

助手 2026-07-10 09:06:05

已经改好。现在

[run_stage2_leaky.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/run_stage2_leaky.py:190)
支持按顺序取泄露样本。

你可以这样用:

```
--leak_train_on_eval \  
--leak_eval_fraction 0.5 \  
--leak_eval_selection first \  
--clean_context
```

****新参数****

我新增了:

```
--leak_eval_selection random|first|last
```

含义:

random:	--leak_eval_seed		
first:	Stage2	val/test	50%
last:	Stage2	val/test	50%

因为 `meta\_df` 在 `prepare\_sorted\_stage2\_data()` 里已经按:

```
flow_start_timestamp_us, flow_id
```

排序, 所以:

```
first = split  
last = split
```

****代码怎么修的****

原来代码是固定随机:

```
leaked_rows = rng.choice(rows, size=n_leak, replace=False)
```

现在我加了一个选择函数:

```
def select_leaked_rows(rows, n_leak, selection, rng):  
    if selection == "random":  
        return rng.choice(rows, size=n_leak, replace=False)  
    if selection == "first":  
        return rows[:n_leak]  
    if selection == "last":  
        return rows[-n_leak:]
```

位置: [run_stage2_leaky.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/run_stage2_leaky.py:233)

然后在泄露 val/test 时调用它:

```
leaked_rows = select_leaked_rows(  
    rows=rows,  
    n_leak=n_leak,  
    selection=leak_eval_selection,  
    rng=rng,  
)
```

位置: [run_stage2_leaky.py](C:/Users/XiaoyanXiong/Desktop/NIDSTransformer/s2/run_stage2_leaky.py:277)

****按顺序取前 50% 的完整命令****

```
!python /content/drive/MyDrive/s2/run_stage2_leaky.py \
  --stage1_dir /content/drive/MyDrive/s1/0704/0704C_ar002_et12_20260511_001 \
  --config /content/drive/MyDrive/s2/0709ar002_et12_20260511_001/target_query_gated_v1/stage2_config_used.
yaml \
  --out_dir /content/drive/MyDrive/s2/LEAKY/half_first_val_test_to_train_clean_context \
  --context_method source_host \
  --context_policy online \
  --window_size 128 \
  --seed 130 \
  --epochs 100 \
  --batch_size 128 \
  --leak_train_on_eval \
  --leak_eval_fraction 0.5 \
  --leak_eval_selection first \
  --clean_context
```

****如果想取后 50%****

```
--leak_eval_selection last
```

****注意****

用了 `first/last` 后, `--leak\_eval\_seed` 对选择样本不再起作用, 因为不随机了。但保留它也不会出错。

我已做过语法检查, 脚本可以正常解析。